

Programación en Móviles Avanzado

LABORATORIO N° 06

ViewControllers – Navegación Modal - Segues



DOCENTE:

JUAN LEON

CURSO:

Programación en Móviles
Avanzado
5 C24

ALUMNO:

JUNIOR CUEVA

REPOSITORIO:

github.com/juniorcueva-star/MovilesAvanzado JuniorCueva

PROGRAMACIÓN EN MÓVILES AVANZADOS : GESTIONAR VISTAS Y NAVEGACIÓN EN UIKIT

I. Capacidades

- Implementar estructuras condicionales y bucles en Swift para solucionar problemas lógicos.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora..

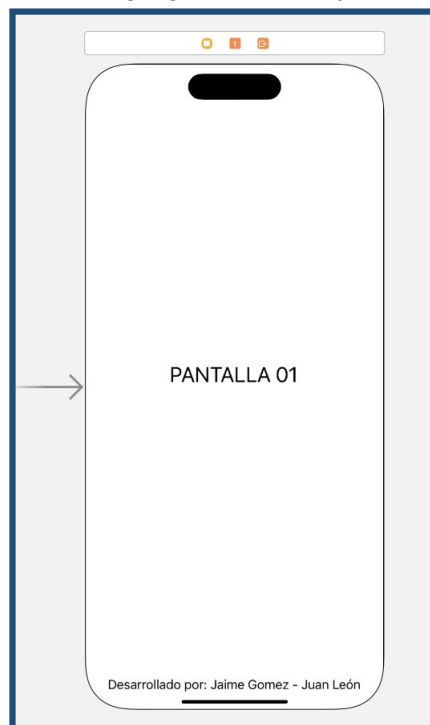
VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es en grupo de a 2 estudiantes

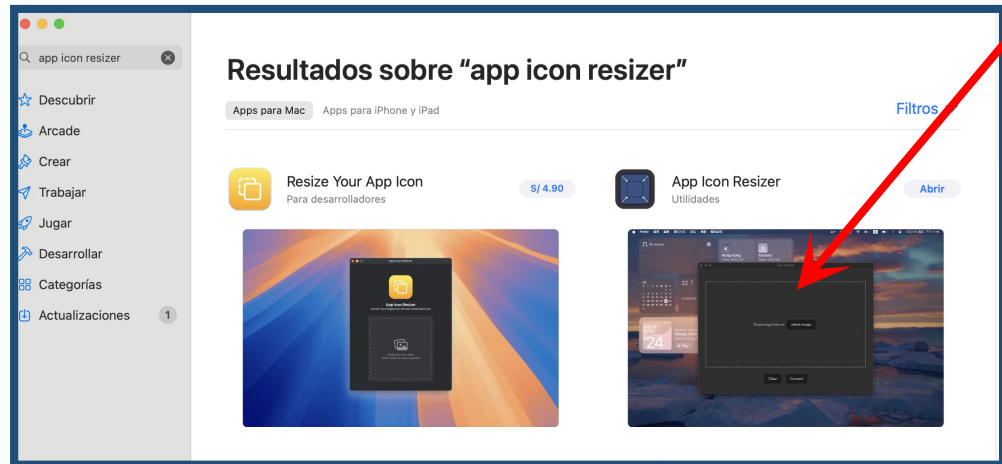
VII.- PROCEDIMIENTO:

PRIMERA PARTE

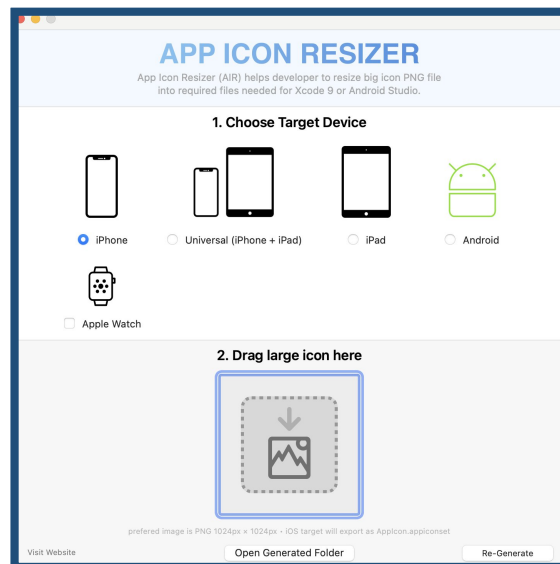
1. Crear un proyecto en UIKit
2. Es la única vista creada en Main, agregar un Label y colocar de texto : PANTALLA 1



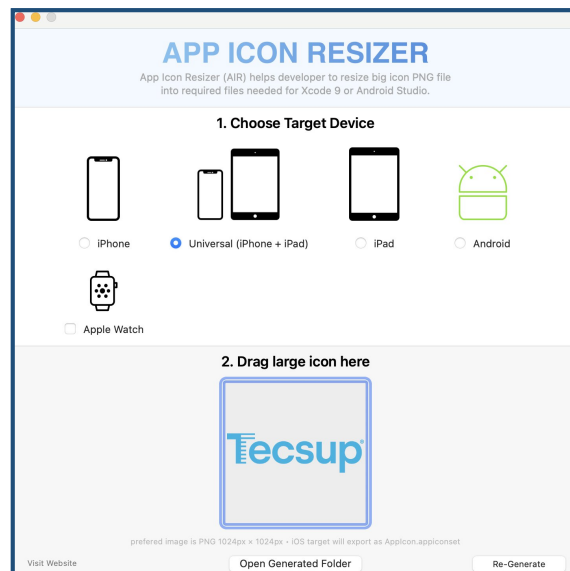
3. Vamos a colocar en primer lugar una imagen como icono de nuestra app, colocaremos el logo de TECSUP como referencia, en formato .png, guardarlo en la carpeta donde está tu aplicación.
4. Verifique en la mac que tiene la App llamada : App Icon Resizer(AIR) en todo caso descarguela de la App Store, en todo caso también tiene la alternativa de utilizar la web: <https://hotpot.ai/icon-resizer>
5. Tenga presente que en el App Store se muestra de esta forma :



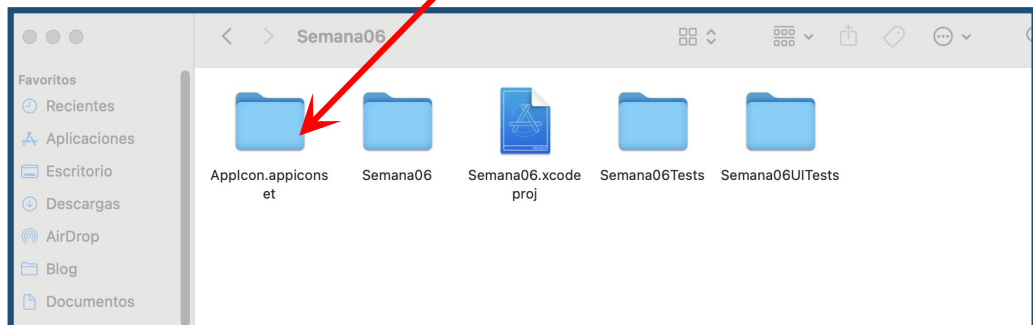
6. Abrir la aplicación :



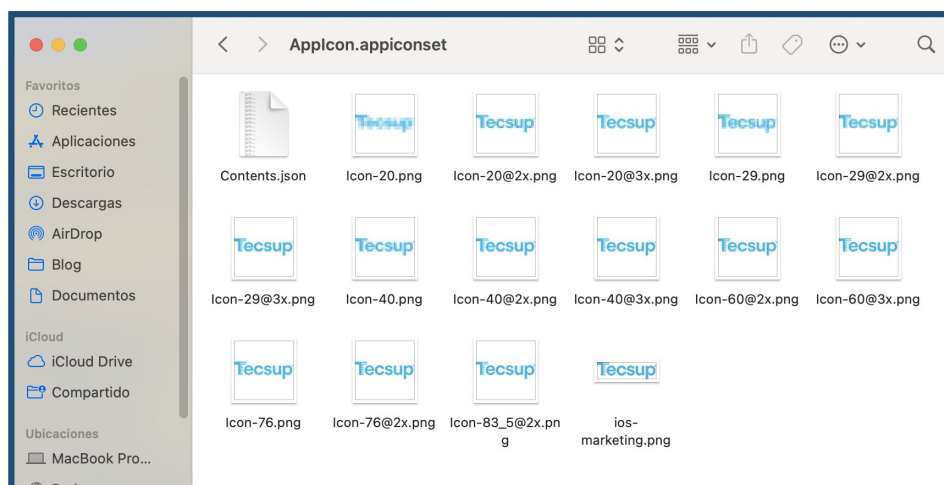
7. Seleccionar la opción Universal y arrastra la imagen y guardarla en la Carpeta donde se encuentra la aplicación



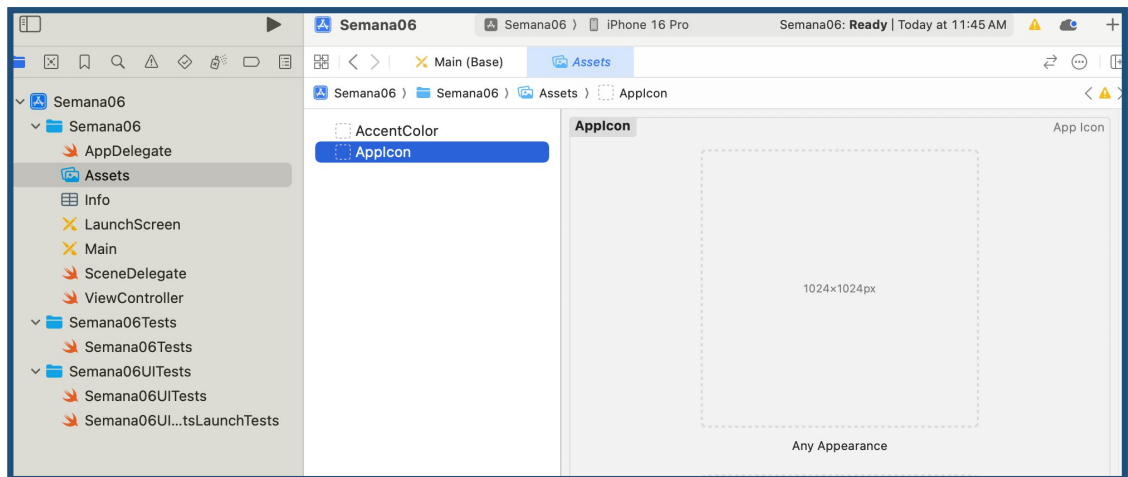
8. Una vez guardada , en la carpeta donde que contiene la aplicación guardada buscar la sub carpeta :AppIcon.appiconset



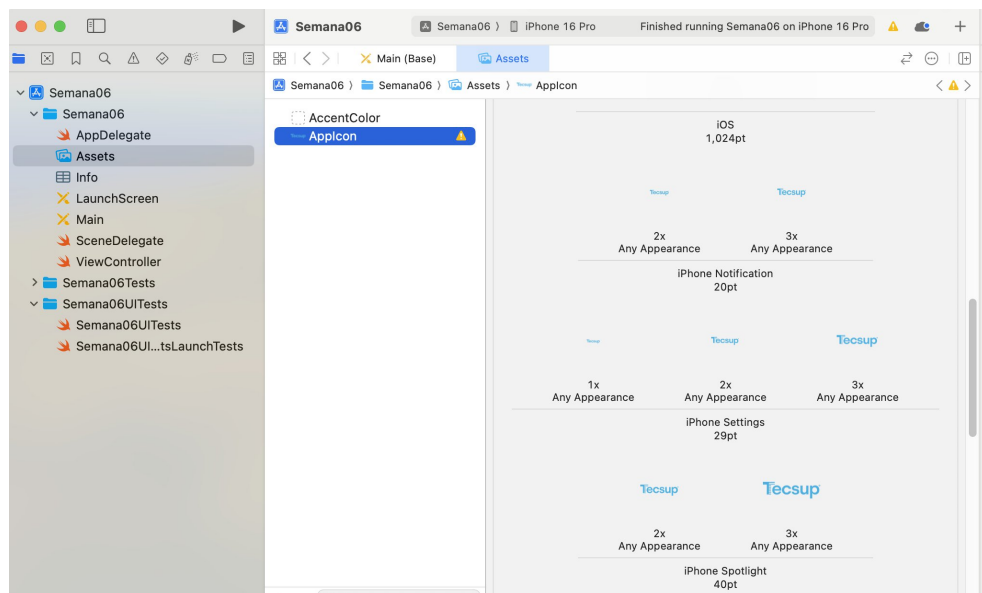
9. Contiene imágenes en las diferentes resoluciones.



10. Diríjase al proyecto en Xcode y haga clic en la carpeta: **Assets** y seleccione **AppIcon**



11. Abra la carpeta Applcon.appiconset y seleccione todos los iconos generados, excepto .Json y arrastre estos iconos sobre su proyecto Xcode, en la lista mostrada por Applcon



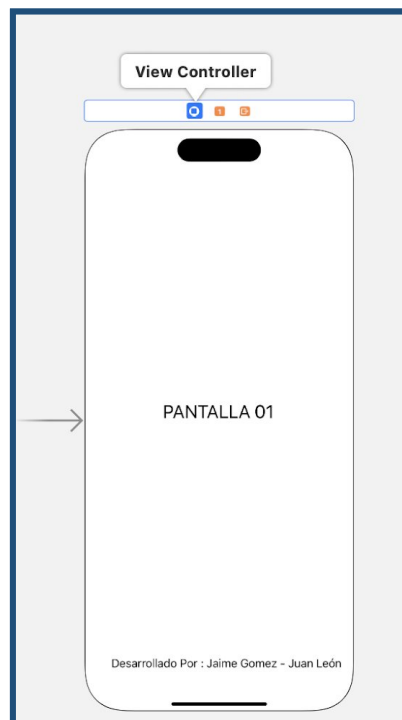
12. Ejecute el proyecto y verifique que se muestre el icono que se agrego.



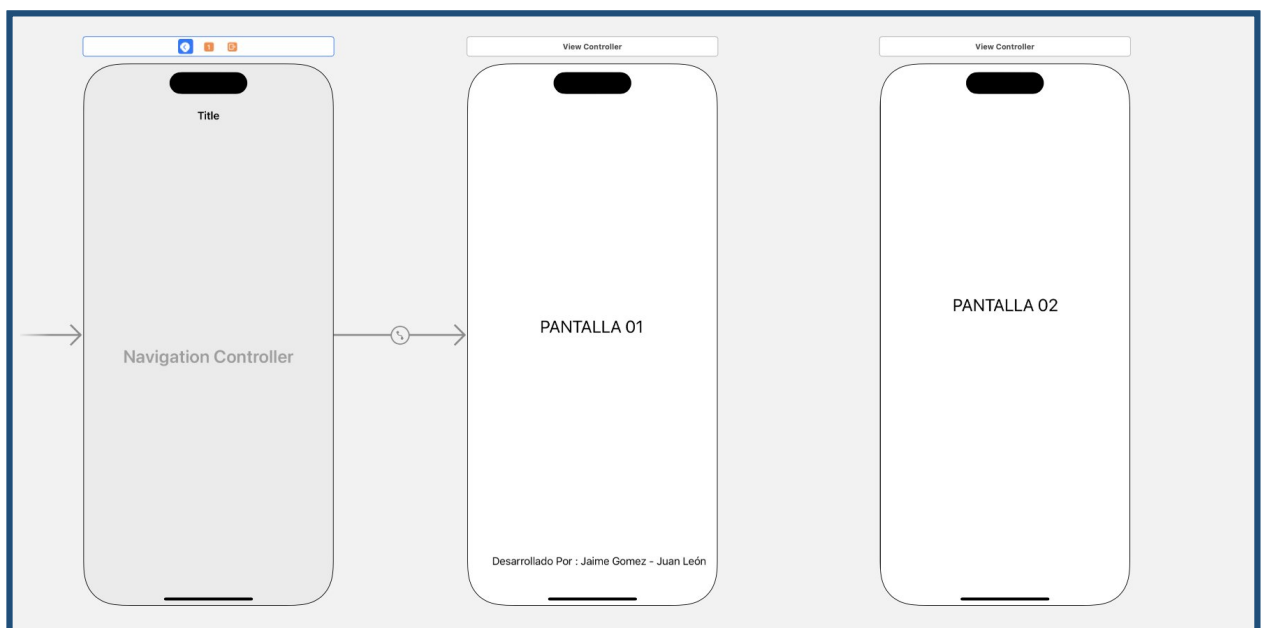
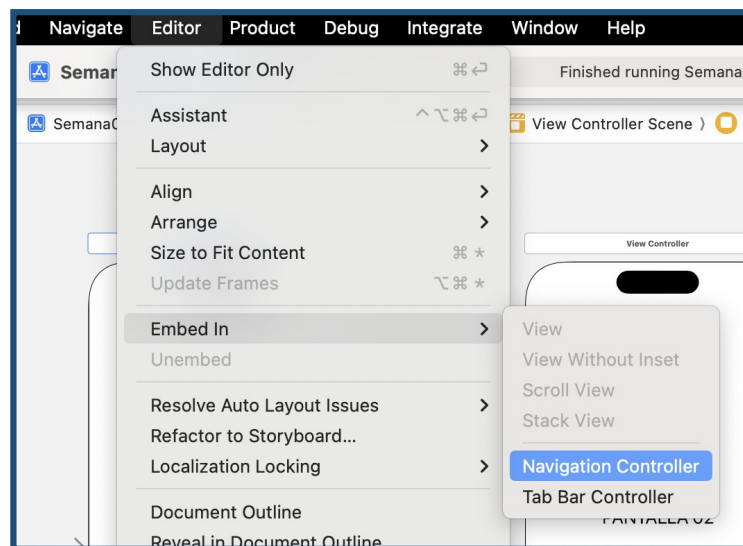
13. Agregar una Segunda Pantalla, buscar en la libreria de objetos a **viewController**, coloque un Label con el nombre PANTALLA 02



14. Seleccione la primera Vista y haga clic en su botón ViewController



15. Una vez seleccionado, diríjase al menú Editor -> Navigation Controller
- Se habrá aumentado una vista de navegación inicial, antes de la primera pantalla



16. Que cambio pudo notar en el diseño de la vista1

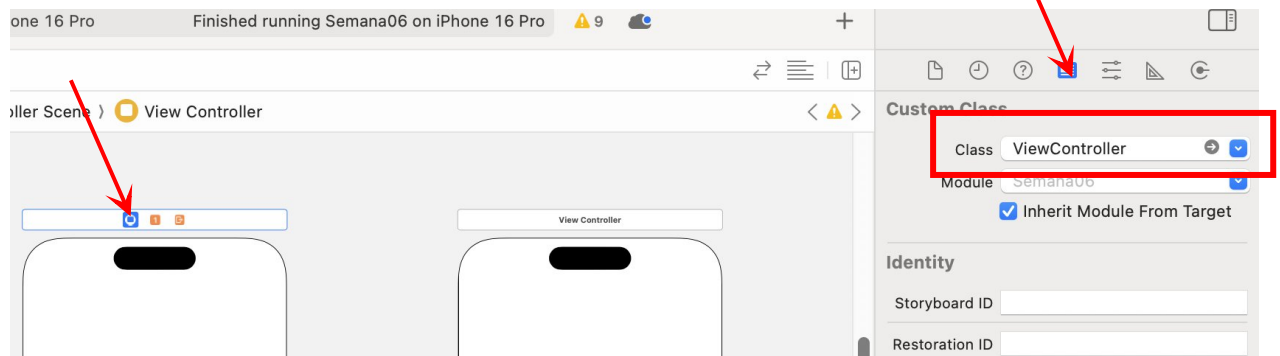
Respuesta: Al embeber la vista 1 en un Navigation Controller se le agregó una barra de navegación (Navigation Bar) en la parte superior, con su Navigation Item para colocar título y botones. Además apareció una escena nueva llamada Navigation Controller, que ahora es el punto de entrada del storyboard (la flecha inicial apunta a ella) y queda unida a la vista 1 con un segue de relación (root view controller).

17. Investigue para que sirve un **Navigation Controller**

Respuesta: Es un controlador contenedor que administra una pila (stack) de pantallas. Cuando se avanza a otra pantalla la apila (push) y cuando se regresa la quita (pop). Entrega de forma automática la barra superior con el título y el botón Back, por lo que sirve para la navegación jerárquica: ir de lo general al detalle y poder volver atrás.

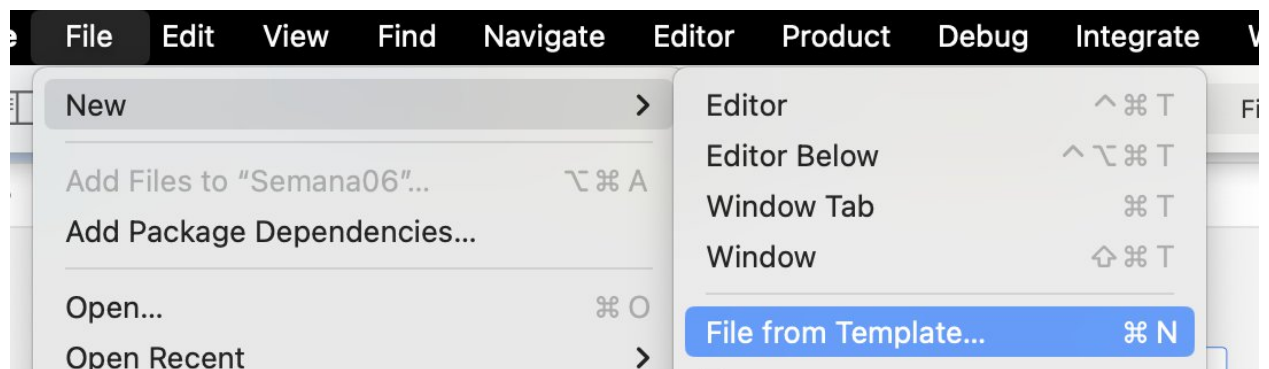
En este laboratorio permite pasar de PANTALLA 1 a PANTALLA 2 con el segue Show y regresar con el botón Back.

18. Seleccione nuevamente el botón del viewController en la primera vista y active la pestaña “Show the Identity Inspector” en la barra lateral derecha

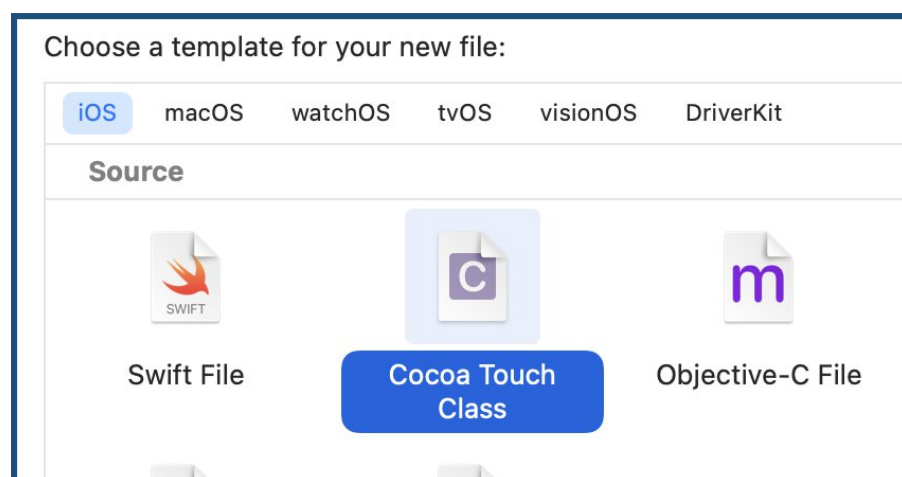


19. Notará que la clase ViewController es la que controla a esta vista(es aqui donde se coloca el código que controlara la vista)deberemos hacer lo miso con la nueva vista creada.

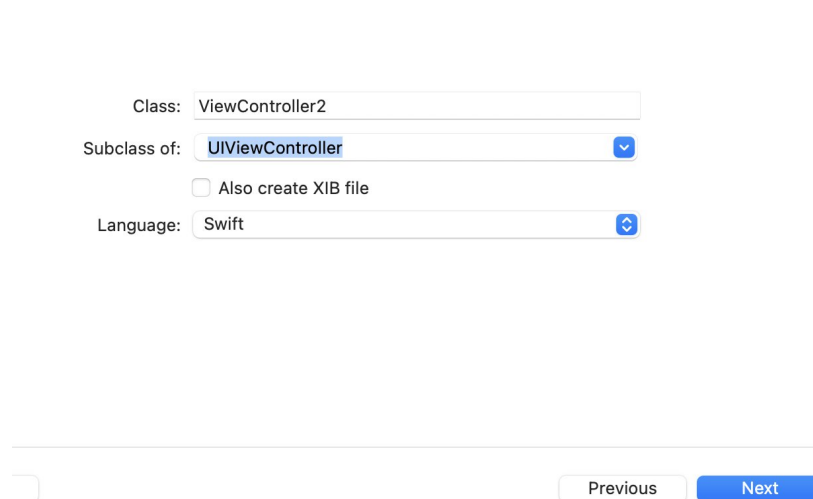
Para ello, agregue un nuevo archivo(File->New->File from Template),el archivo debe ser de tipo Cocoa Touch y el nombre del archivo es **ViewController2**



20. Seleccionar Cocoa Touch Class

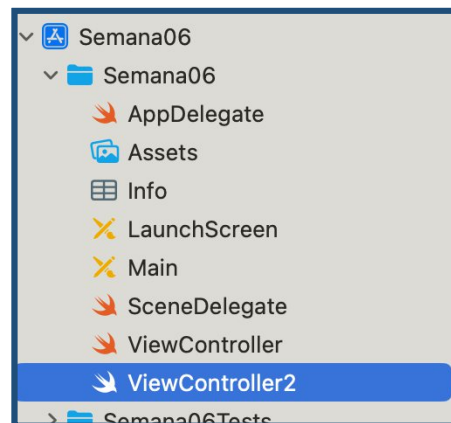


21. Después de definir el nombre , debe ser una sub clase de **UIViewController** y con **lenguaje swift** . Además el check box de “Also create XIB file” debe estar sin marcar.



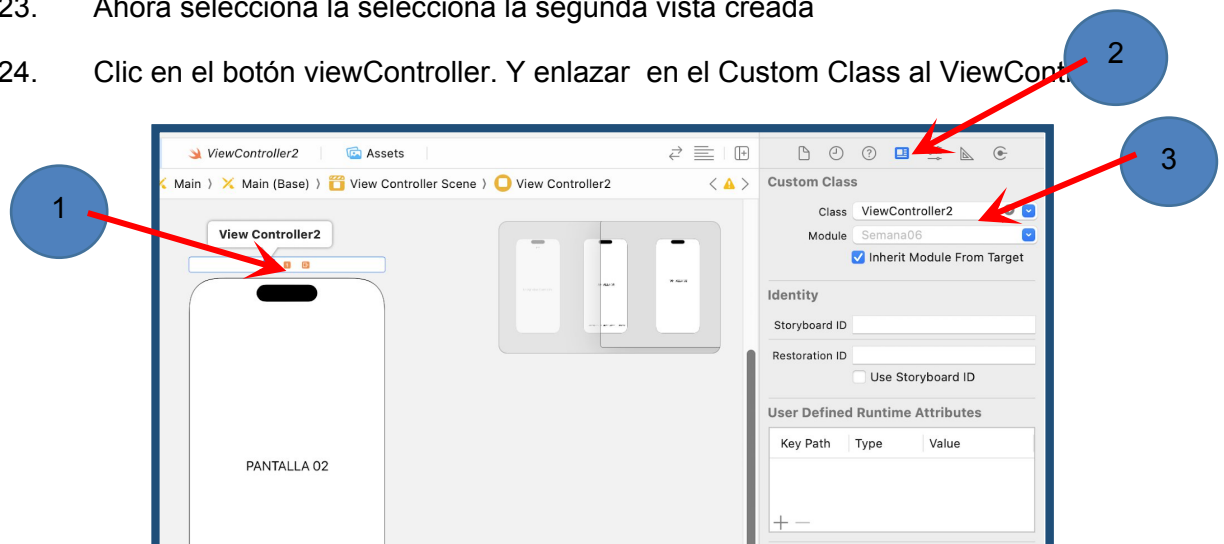
The screenshot shows the 'New' dialog in Xcode. The 'Class' field is set to 'ViewController2'. The 'Subclass of' dropdown is set to 'UIViewController'. The 'Also create XIB file' checkbox is unchecked. The 'Language' dropdown is set to 'Swift'. At the bottom, there are 'Previous' and 'Next' buttons.

22. Se mostrará la siguiente estructura.



23. Ahora selecciona la selecciona la segunda vista creada

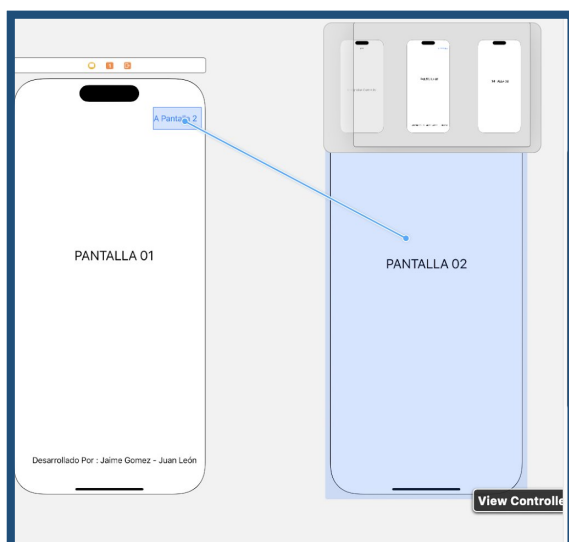
24. Clic en el botón viewController. Y enlazar en el Custom Class al ViewConti



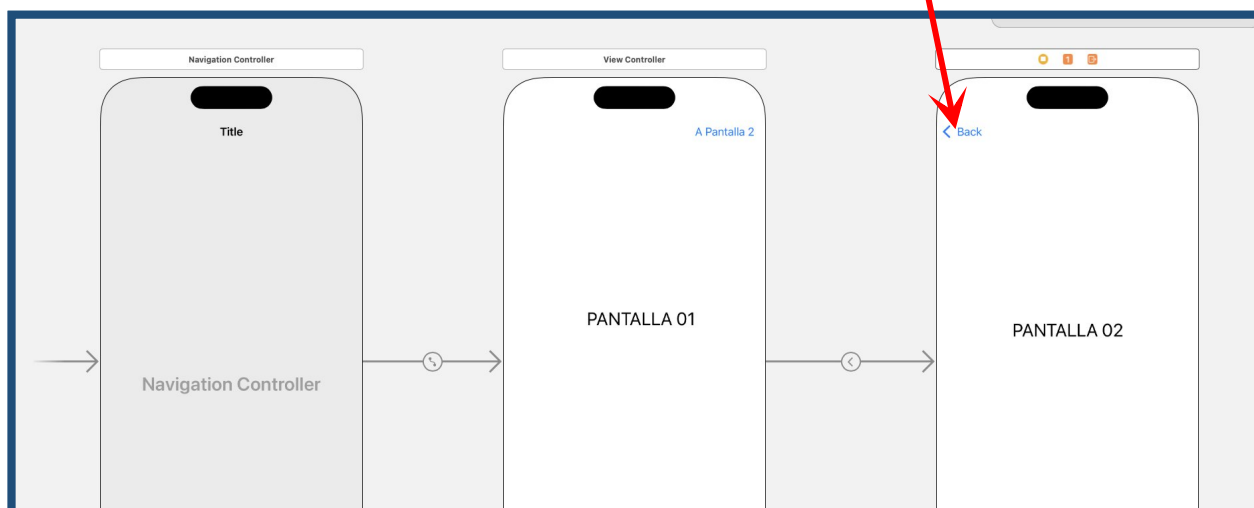
25. En el cuadro búsqueda de controles busque el Control Bar Button Item y arrástralo a la Vista 1, a continuación cambie el nombre del texto del botón por **A Pantalla 2**



26. Ahora mantenga presionada la tecla Ctrl y con clic arrastre el enlace hasta la pantalla 2, del menú contextual seleccione la opción **Show**



27. Tendrás esta presentación :



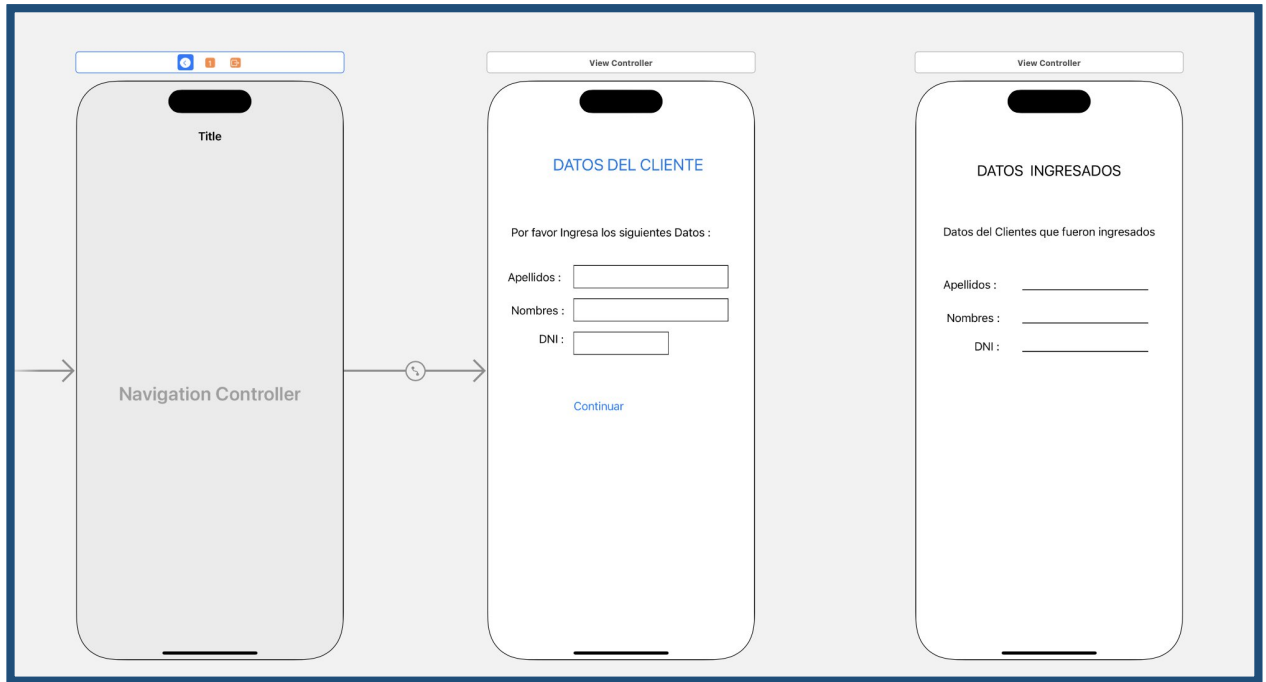
28. Ejecute la aplicación

Evidencia: la app se ejecuta y navega de PANTALLA 1 a PANTALLA 2. Las capturas y la respuesta de la pregunta 29 están en la página siguiente.

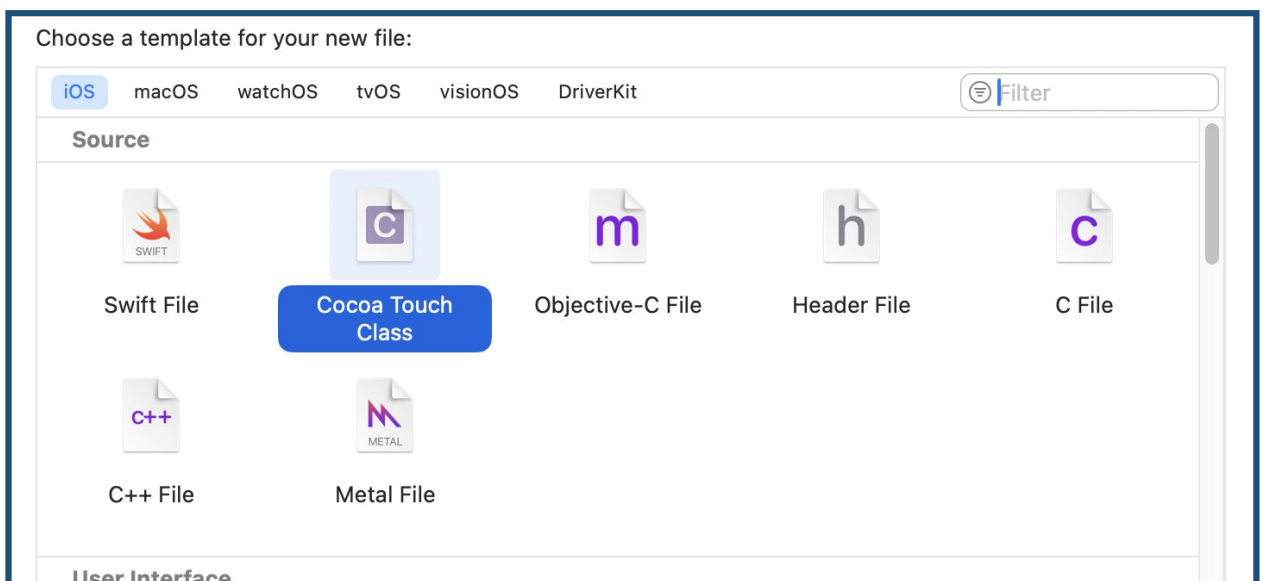
29. Investigue para qué sirven las opciones : Show, Show Detail, Present Modally, Present As Popover.

VENTANAS MODALES

1. Crear una aplicación de nombre Semana06_02
2. Realizar una App con la siguiente presentación.



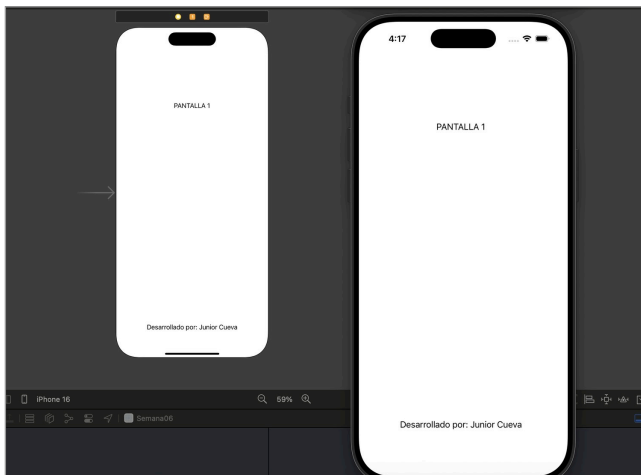
3. Agregar una clase :Cocoa Touch Class



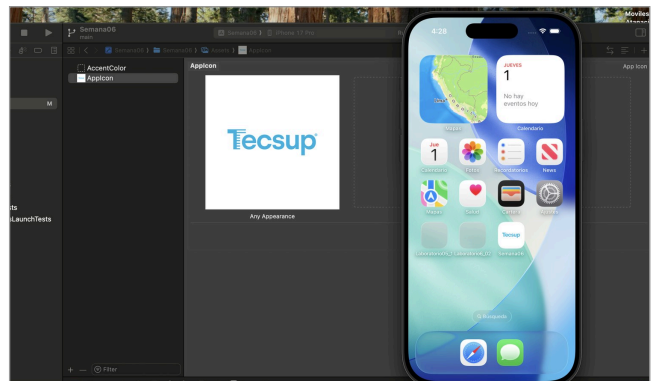
4. Definir el Nombre de la clase y el tipo de sub-clase

Evidencias de la Primera Parte y respuesta de la pregunta 29

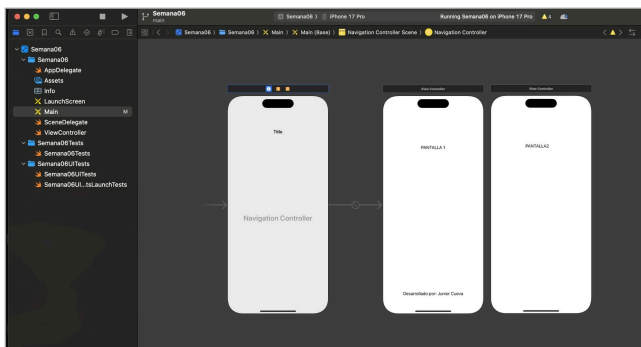
Proyecto Semana06 (rama main) - https://github.com/juniorcueva-star/MovilesAvanzado_JuniorCueva



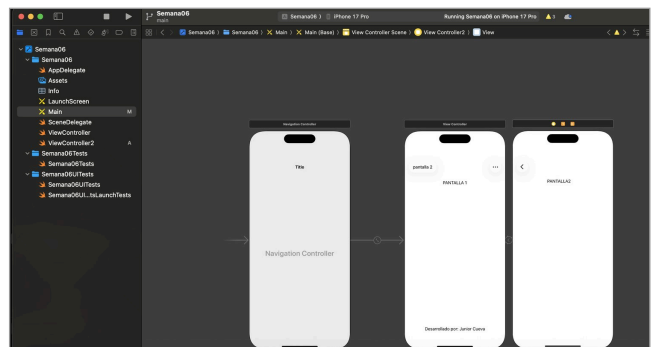
Pasos 1 y 2: PANTALLA 1 en el storyboard y ejecutándose en el simulador



Pasos 10 al 12: logo en Assets > Applcon e icono Semana06 en la pantalla de inicio



Pasos 13 al 15: Navigation Controller agregado antes de la vista 1



Pasos 25 al 27: Bar Button Item con segue Show y botón Back en la pantalla 2

Respuesta 29. ¿Para qué sirven Show, Show Detail, Present Modally y Present As Popover?

Show: avanza a la siguiente pantalla. Si hay Navigation Controller la apila (push) y aparece el botón Back; si no lo hay, la presenta de forma modal.

Show Detail: reemplaza el panel de detalle de un Split View Controller (lista a la izquierda, detalle a la derecha). En iPhone se comporta como un Show.

Present Modally: presenta la pantalla encima de la actual e interrumpe el flujo. No hay botón Back: se cierra con dismiss. Es la que se usa en el ejercicio de Ventanas Modales.

Present As Popover: muestra la pantalla como una burbuja flotante anclada a un botón. Tiene sentido en iPad; en iPhone se adapta y se ve como una hoja modal.

Class: ClienteModel

Subclass of: NSObject

☐ Also create XIB file

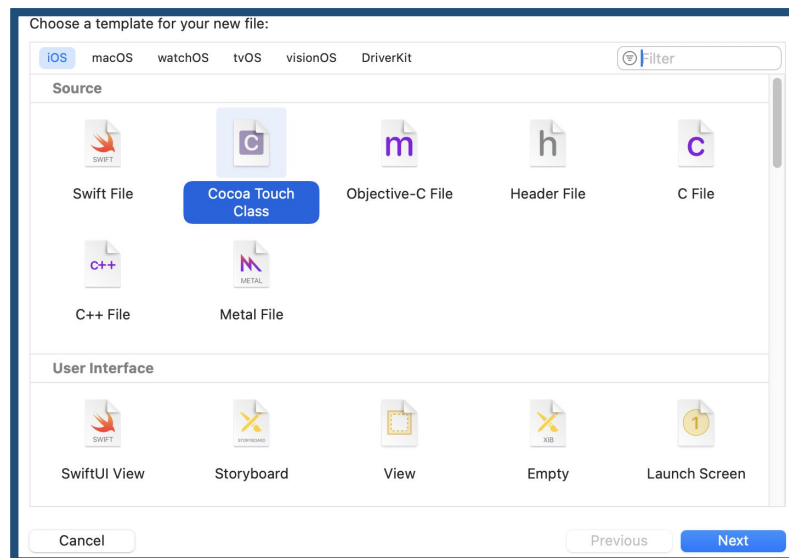
Language: Swift

Previous Next

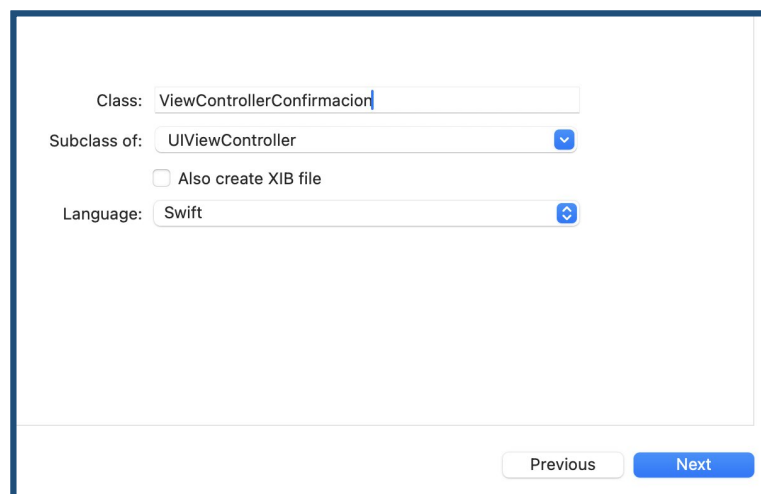
5. Definir el siguiente código en la clase creada

```
2  import UIKit
3  class ClienteModel: NSObject {
4      varCodigo: Int32 = 0
5      varApellido: String = ""
6      varNombre: String = ""
7      varDni: String = ""
8      // inicializador sin parametros y con Parametros
9      override init()
10     {
11         self.Codigo = 0
12         self.Apellido = ""
13         self.Nombre = ""
14         self.Dni = ""
15     }
16     init(pCodigo: Int32,pApellido: String,pNombre: String,pDni: String)
17     {
18         self.Codigo = pCodigo
19         self.Apellido = pApellido
20         self.Nombre = pNombre
21         self.Dni = pDni
22     }
```

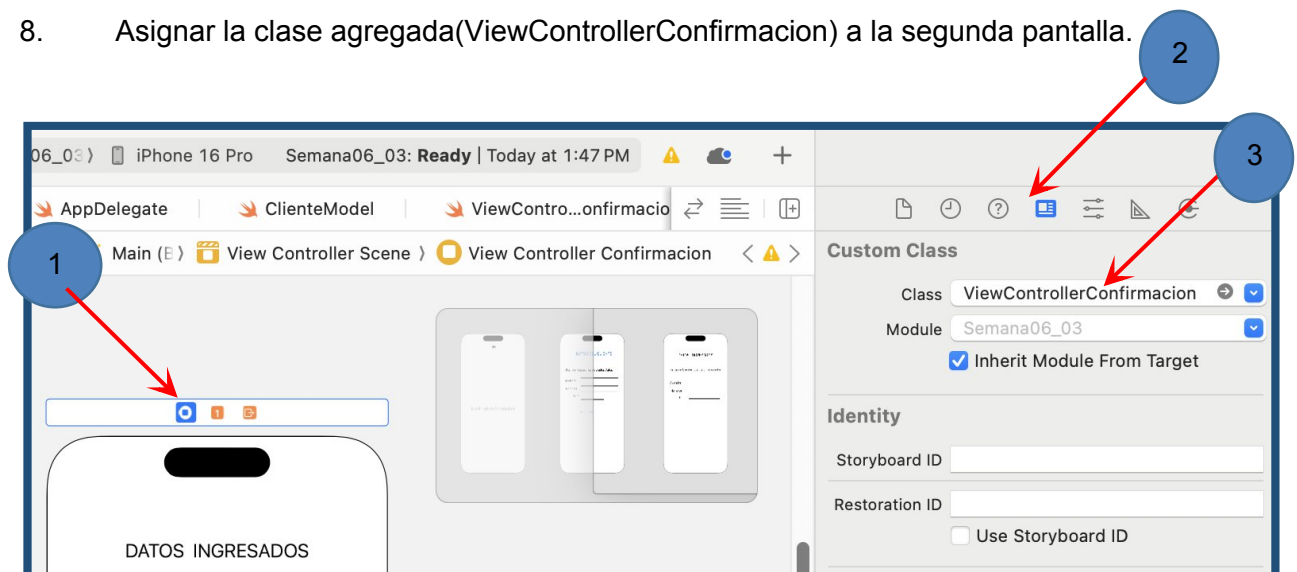
6. Crearemos un controlador para la segunda pantalla



7. Colocar los siguientes nombres : **ViewControllerConfirmacion**

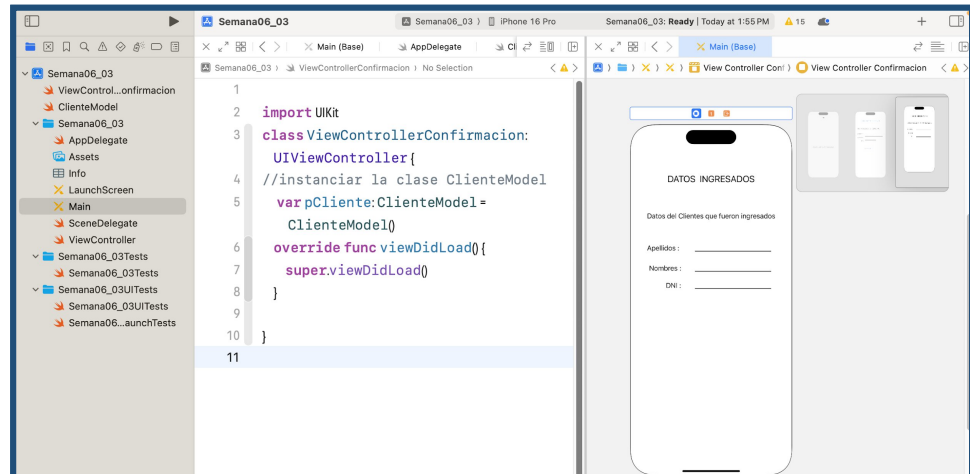


8. Asignar la clase agregada(ViewControllerConfirmacion) a la segunda pantalla.

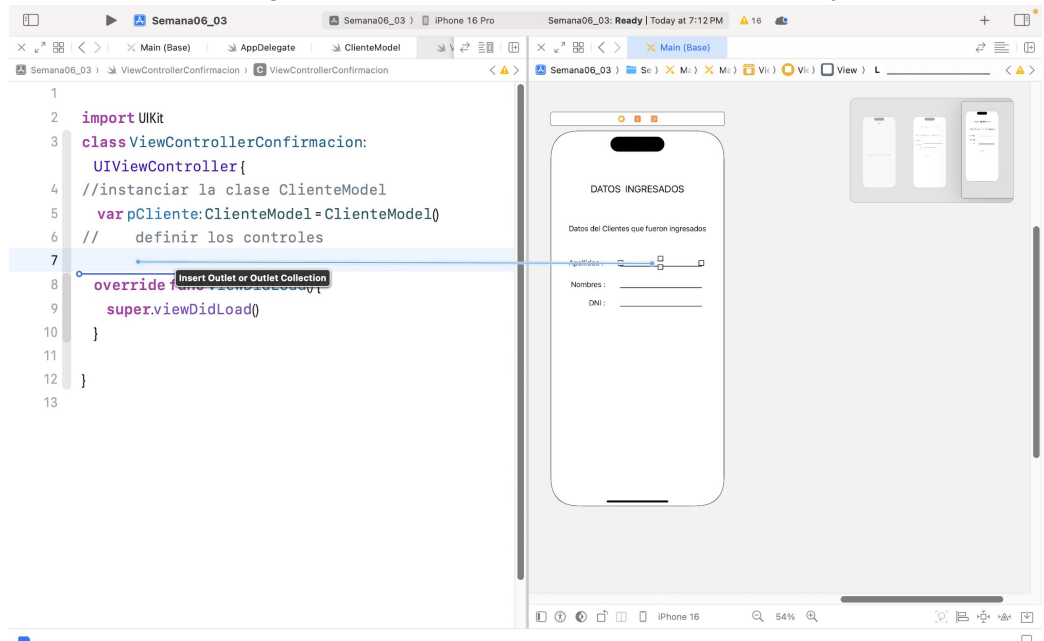


9. En el. Controlador agregado en el paso 7 (ViewControllerConfirmacion)

Realizar el siguiente código, para ello debe de tener en su pantalla la siguiente vista.



10. Definir el código de la instancia de la clase ClienteModel y de los controles

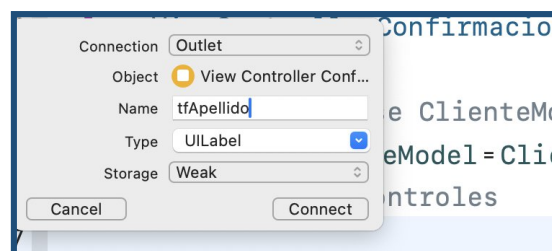


Colocar los nombres de los controles :

tfApellido

tfNombre

tfDni



```

UIViewController{
//instanciar la clase ClienteModel
var pCliente:ClienteModel=ClienteModel()
//    definir los controles
@IBOutlet weak var tfApellido:UILabel!
@IBOutlet weak var tfNombre:UILabel!
@IBOutlet weak var tfDni:UILabel!
override func viewDidLoad(){
    super.viewDidLoad()
}
}

```

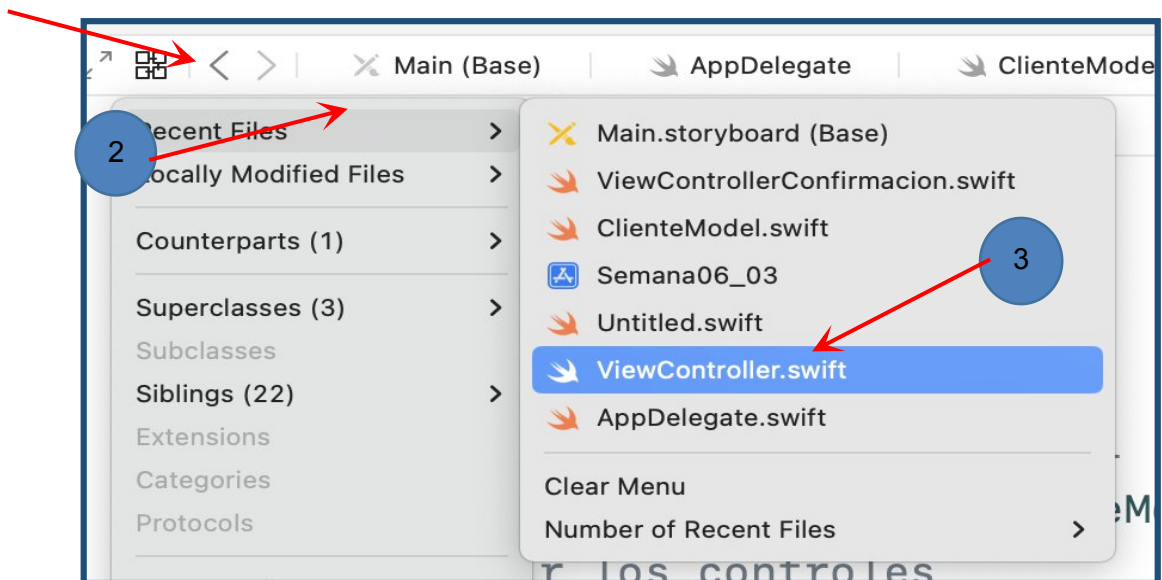
Complementar el código :

```

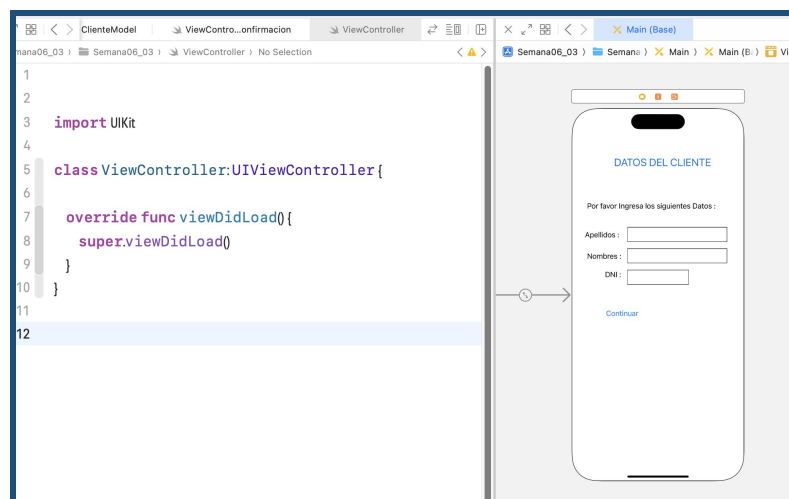
import UIKit
class ViewControllerConfirmacion:
    UIViewController{
//instanciar la clase ClienteModel
var pCliente:ClienteModel=ClienteModel()
//    definir los controles
@IBOutlet weak var tfApellido:UILabel!
@IBOutlet weak var tfNombre:UILabel!
@IBOutlet weak var tfDni:UILabel!
override func viewDidLoad(){
    super.viewDidLoad()
//    definir los controles
self.tfApellido.text=pCliente.Apellido
self.tfNombre.text=pCliente.Nombre
self.tfDni.text=pCliente.Dni
}
}

```

Continuar con la primera vista , previamente llamar **viewController.swift**



Para ello debemos tener la siguiente vista



Realizar la actividad de llevar los controles al código y tener esta presentación

```
import UIKit

class ViewController: UIViewController {

    @IBOutlet weak var tfApellido: UITextField!
    @IBOutlet weak var tfNombre: UITextField!
    @IBOutlet weak var tfDni: UITextField!
    override func viewDidLoad() {
        super.viewDidLoad()
    }
    @IBAction func btnContinuar(_ sender: Any) {
    }
}
```

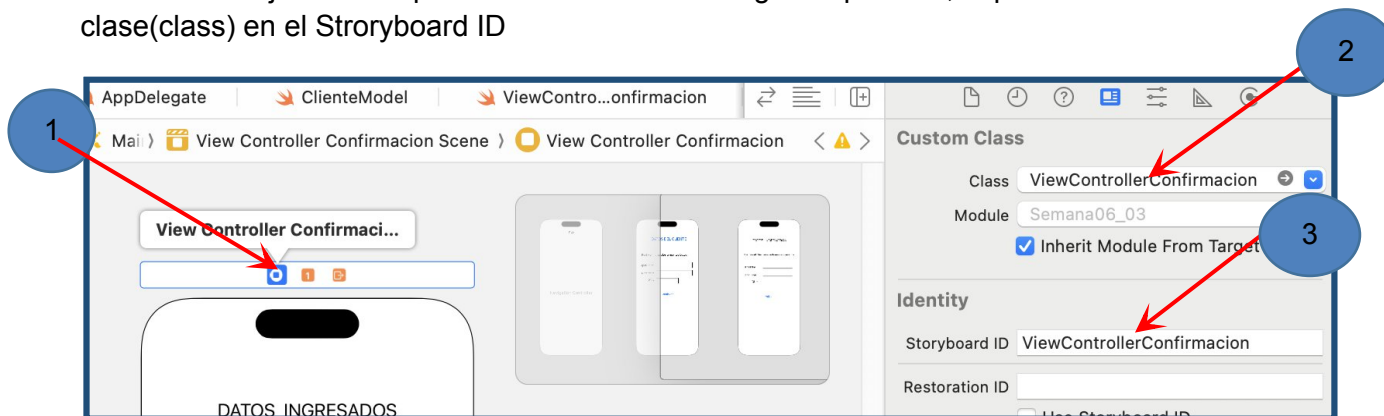
Complementar el código:

```

@IBAction func btnContinuar(_ sender: Any) {
    let oCliente: ClienteModel = ClienteModel(pCodigo: 0, pApellido: self.tfApellido.text!,
                                                pNombre: self.tfNombre.text!, pDni: self.tfDni.text!)
    //      crear el Storyboard con el nombre Main, se asocia a la pantalla 2
    //      le pasamos los datos del cliente
    let osb : UIStoryboard = UIStoryboard(name: "Main", bundle: nil)
    let oPantalla2 = osb.instantiateViewController(identifier: "ViewControllerConfirmacion") as!
        ViewControllerConfirmacion
    oPantalla2.pCliente = oCliente
    self.present(oPantalla2, animated: true, completion: nil)
}
}

```

11. Antes de ejecutar la aplicación seleccionar la segunda pantalla, copiar el nombre de la clase(class) en el Storyboard ID



12. Ejecutar la aplicación.

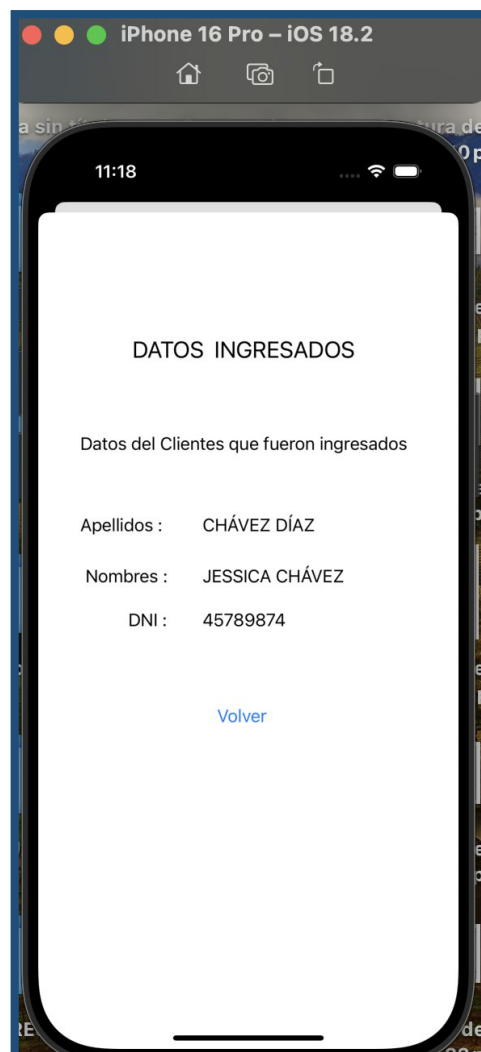
Evidencia: ejecución de Semana06_02 en el simulador (iPhone 17 Pro).



Formulario DATOS DEL CLIENTE



Ventana modal DATOS INGRESADOS



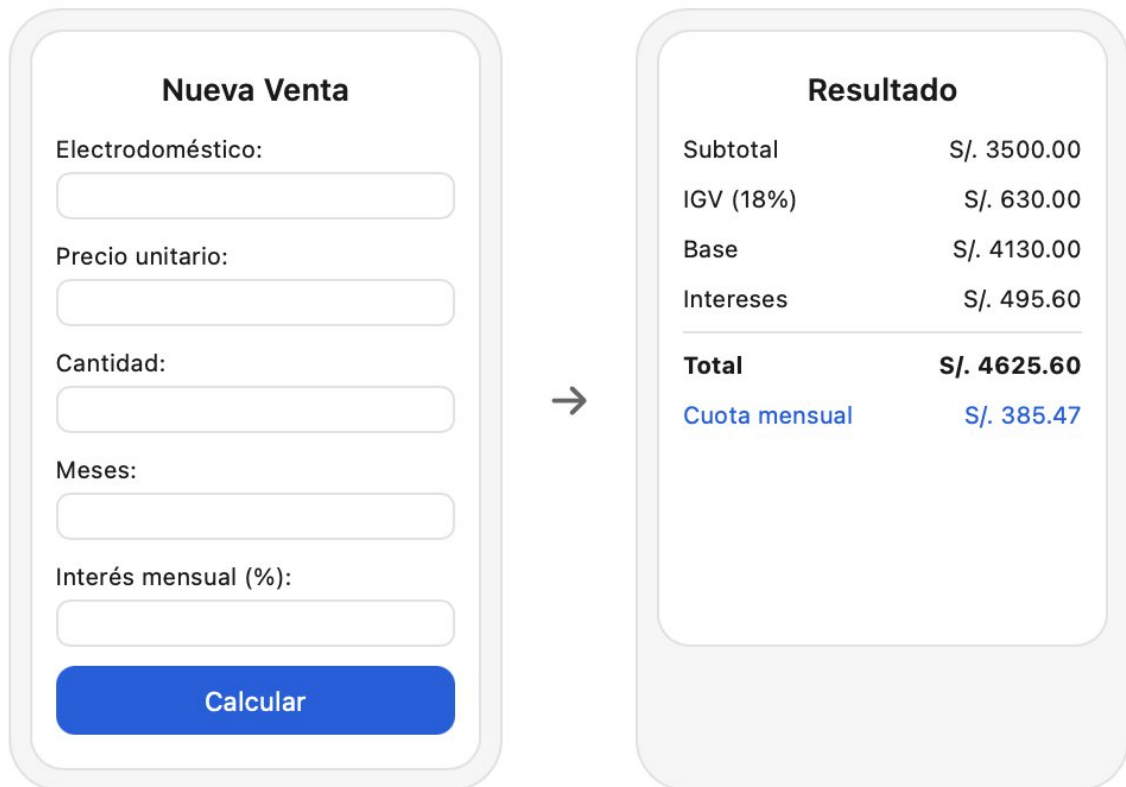
Calculadora de Venta a Plazos de Electrodoméstico

Rama ai-assisted, con estructura CTRFE

Por qué es el cierre lógico: junta todo lo anterior — Show, un modelo propio (esta vez VentaModel), y formato de datos — en un caso más completo y realista, con menos guía paso a paso a propósito.

Vista (diseñala tú en el Storyboard):

- Pantalla "Nueva Venta": campos para nombre del electrodoméstico, precio unitario, cantidad, número de meses y tasa de interés mensual, más un botón "Calcular".
- Pantalla "Resultado": muestra subtotal, IGV, monto base, intereses totales, total a pagar y cuota mensual — todo formateado en soles.



Pasos paso a paso en Interface Builder (con menos guía — ya conoces el patrón)

1. Arrastra una nueva pantalla al Storyboard para "Nueva Venta" (o reutiliza una existente) y otra para "Resultado".
2. En "Nueva Venta", agrega 5 UITextField (uno por cada dato de entrada) con su UILabelcorrespondiente, y un botón "Calcular".
3. Crea el segue **Show** desde "Calcular" hasta "Resultado", con identifier showResultado.
4. Crea las clases Swift para ambas pantallas y asígnalas como Custom Class en el Inspector de Identidad.
5. Conecta los 5 UITextField como IBOutlet en la pantalla "Nueva Venta".
6. En "Resultado", agrega 6 UILabel (uno por cada valor calculado) y conéctalos como IBOutlet.
7. Implementa el cálculo y el prepare(for:sender:) usando VentaModel, siguiendo el mismo patrón del anterior

Fórmulas exactas a implementar

```
subtotal = precioUnitario x cantidad
igv      = subtotal x 0.18
base     = subtotal + igv
intereses = base x (tasaInteresMensual / 100) x meses
total    = base + intereses
cuota    = total / meses
```

Enunciado a pedirle a la IA

1. Define class VentaModel: NSObject con propiedades para las 6 salidas del cálculo (subtotal, igv, base, intereses, total, cuota), todas Double.
2. En la pantalla "Nueva Venta", implementa el cálculo completo con las fórmulas de arriba y arma un VentaModel con los resultados.
3. Pasa el VentaModel a la pantalla "Resultado" con prepare(for:sender:), igual que en el Ejercicio 2.
4. En "Resultado", muestra cada valor formateado como soles: String(format: "S/. %.2f", valor).

Restricciones para el prompt (CTRFE)

- Solo lo visto hasta semana 6: clases, UINavigationController, prepare(for:sender:), IBOutlet/IBAction.
- Nada de Combine, Codable, ni persistencia — el objetivo es reforzar paso de datos con un modelo propio, no aprender temas nuevos.
- Pide a la IA que explique por qué usa class y no struct para VentaModel, dado lo visto en el PREDICT del Ejercicio 2.

↑ Sube tu avance: Commit → "feat: Ejercicio 4 - calculadora de venta a plazos (IA)" → Push

Hecho: [commit 4eae636](#) "feat: Ejercicio 4 - calculadora de venta a plazos (IA)" subido (push) a la rama ai-lab06.

PROMPTS.md (rama ai-assisted)

Mismo formato de labs anteriores: Contexto → Tarea → Restricciones → Formato → Ejemplo, más la reflexión de qué hizo distinto la IA (por ejemplo, si validó los campos vacíos sin que se lo pidieras, o si usó guard let en vez del patrón visto en clase).

Entregado: [semana 06/PROMPTS.md](#) en la rama ai-lab06 (prompt CTRFE y reflexión).

Conclusiones

1. ¿Cuándo conviene usar Show y cuándo Present Modally? Da un ejemplo real de app donde usarías cada uno.
2. Investiga para qué sirven Show Detail y Present As Popover y en qué dispositivo tienen sentido.
3. ¿Qué pasaría si ClienteModel o VentaModel fueran struct en vez de class? ¿Se rompería el paso de datos hacia adelante?
4. ¿Qué diferencia notaste entre resolver el Ejercicio 2 (manual) y el Ejercicio 4 (con IA) en tiempo y comprensión?

Respuestas:

1. Show conviene cuando la pantalla nueva es el siguiente nivel de la misma información y el usuario necesita volver atrás; ejemplo: en una app de tienda, pasar de la lista de productos al detalle de un producto. Present Modally conviene para una tarea puntual que interrumpe el flujo y se termina o se cancela; ejemplo: redactar un correo nuevo en Mail.

2. Show Detail reemplaza el panel de detalle de un Split View Controller y Present As Popover muestra una burbuja anclada a un control. Tienen sentido en iPad (pantalla ancha); en iPhone el sistema los adapta y se ven como un Show o una hoja modal.

3. No se rompería: la pantalla 2 recibiría una copia con los mismos valores y los mostraría igual. Lo que cambia es que un struct es un tipo por valor: no puede heredar de NSObject y, si la pantalla 2 lo modifica, la pantalla 1 no ve el cambio. Con class ambas comparten la misma referencia.

4. El Ejercicio 2 (manual) tomó más tiempo, pero deja claro qué hace cada conexión y dónde falla cuando algo se olvida. El Ejercicio 4 (con IA) salió más rápido y con extras (valida campos vacíos, evita dividir entre cero), pero exige leer y verificar el código generado.

Evidencias: Calculadora de Venta a Plazos (Ejercicio 4)

Repositorio: https://github.com/juniorcueva-star/MovilesAvanzado_JuniorCueva

Rama ai-lab06, carpeta semana 06/CalculadoraVenta. VentaModel (class, NSObject) se pasa a Resultado con prepare(for:sender:) y el segue Show "showResultado".

Pantalla Nueva Venta (diseño final)

Pantalla Resultado: batidora, precio 600, cantidad 2, 12 meses, 5 %

Comprobación con las fórmulas (batidora)

$\text{subtotal} = 600 \times 2 = 1200.00$

$\text{igv} = 1200 \times 0.18 = 216.00$

$\text{base} = 1200 + 216 = 1416.00$

$\text{intereses} = 1416 \times (5 / 100) \times 12 = 849.60$

$\text{total} = 1416 + 849.60 = 2265.60$

$\text{cuota} = 2265.60 / 12 = 188.80$

¿Por qué class y no struct? VentaModel se crea en Nueva Venta y se entrega a Resultado: al ser class ambas pantallas apuntan al mismo objeto (no se copia) y puede heredar de NSObject, igual que ClienteModel en el Ejercicio 2. Un struct no puede heredar de NSObject y se copia en cada asignación.

Pruebas de cálculo adicionales (primera versión del diseño)

Parlatante: 50 x 10, 2 meses, 4 %. Total S/. 637.20, cuota S/. 318.60

Teclado: 150 x 4, 6 meses, 6 %. Total S/. 962.88, cuota S/. 160.48

Secadora: 80 x 3, 4 meses, 10 %. Total S/. 396.48, cuota S/. 99.12

Commits del laboratorio (GitHub)

main: 7c45be6 proyecto base con pantalla 1 · 3c0a7bf icono de la app · 2cf0771 navegación con segue Show · cb94e4a fix botón A Pantalla 2 · 483c8f4 Semana06_02 y ClienteModel · 77a2bdc diseño de pantallas y outlets · 9b4c8b6 pantalla de confirmación y botón Volver · bd6a4ae paso de datos con ventana modal · 9ef26d3 respuestas y conclusiones.

ai-lab06: f0d0f0c CalculadoraVenta y VentaModel · e7cae23 pantallas y segue showResultado · 2cdbfdb Resultado en soles · 4eae636 calculadora de venta a plazos (IA) · 95ab7e0 PROMPTS.md · 4b706c5 mejora de diseño de la calculadora.